

Distributed Inference on xv6-RISC-V

Milestone 1: Understand and Characterise

Team Information

Abdullah Siraj Khan	31684
Shaheer Ahmed	31543
Muhammad Shayan Naveed	30714
Jawwad Hussain	30676

Repository Link:

https://github.com/Abdullahsirajkhan/OS_Project_XV6_Extended

Division of Work

Team Member	ERP	Task Allocation
Abdullah Siraj Khan	31684	Reverse-engineering of kernel binaries, subsystem architecture mapping for networking/security, and syscall difference analysis.
Shaheer Ahmed	31543	System bring-up and configuration, authoring the execution runbook, and dependency audits.
Muhammad Shayan Naveed	30714	Architectural modeling and Mermaid diagrams, related systems analysis, and framework comparison.
Jawwad Hussain	30676	Baseline performance measurements, codebase inspection, anomaly tracking, and observations log.

September 2026

1 Warm-Up (Section 3.0)

Call Graph Reconstruction

The complete call tree rooted at `compute` (0x101ea) was recovered statically by analyzing RISC-V jump-and-link instructions (`jal ra, offset`) across the text section. First, `compute` branches to two immediate downstream routines: `foo` (0x10160) via `jal` at 0x101fe, and `bar` (0x101a6) via `jal` at 0x1021a. In turn, `foo` invokes the arithmetic leaf function `helper` (0x1013e) at 0x10174 and the Fibonacci routine `fib` (0x100e8) at 0x1018e, establishing the reachable function boundaries. Second, `fib` is verified as recursive because its body contains two self-referential branch calls targeting 0x100e8 at addresses 0x10116 and 0x10128. Third, `foo` is confirmed to be the function called from two distinct places, as it is targeted by `compute` at address 0x101fe and twice by `bar` at addresses 0x101c4 and 0x101d2.

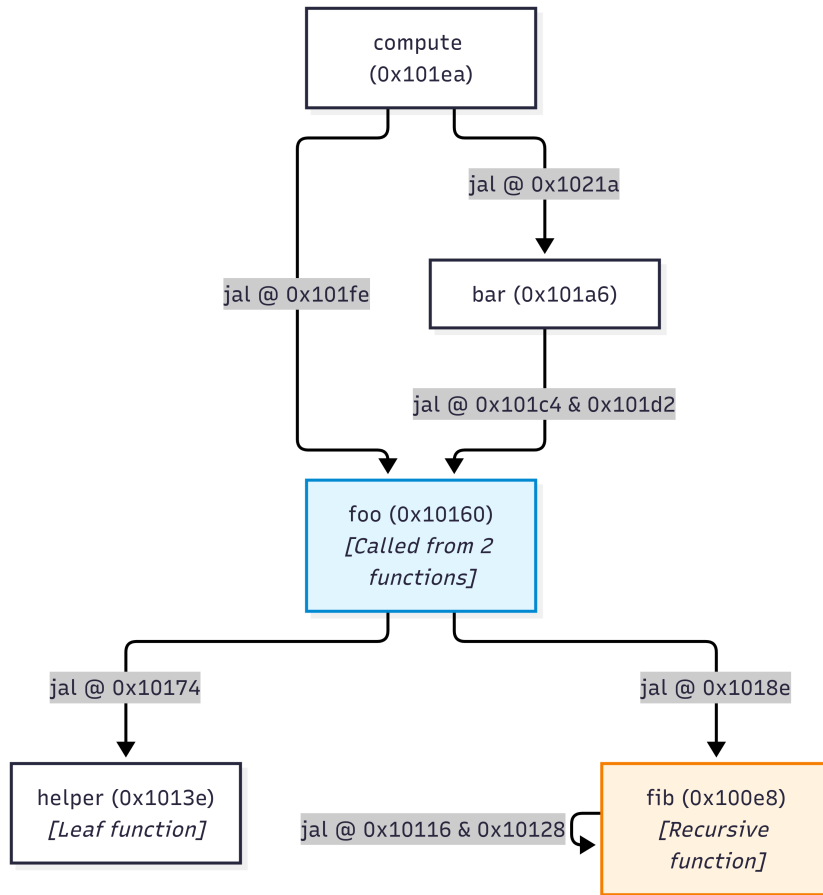


Figure 1: Recovered static call tree rooted at `compute` showing reachable boundaries, self-referential recursive jumps, and multi-targeted leaf functions.

Kernel Entry-Point Trace

To trace a live kernel entry point without kernel C source, GDB was attached over QEMU’s remote stub (`localhost:26000`) and a breakpoint was placed on `sys_exec` at address 0x8000584e. Triggering an execution inside xv6 causes user space to issue an `ecall`, transferring execution through the trampoline vector at virtual address 0x3ffff124 directly into the kernel trap handler `usertrap` (0x80001224). As shown by the backtrace, `usertrap` hands control over to the system call dispatcher `syscall` (0x800014dc), which indexes the system call vector and enters `sys_exec` at frame #0. Inspecting the argument registers at the breakpoint reveals `a0 =`

0x80318d00 (pointing to the executable path string), a1 = 0x3010 (the user-space pointer array for arguments), and a2 = 0x1. Executing `stepi` advances the program counter by two bytes to 0x80005850, confirming single-step instruction execution inside the compiled kernel binary.

```

<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from xv6-riscv/kernel/kernel...
(gdb) target remote localhost:26000
Remote debugging using localhost:26000
0x0000000000000000 in ?? ()
(gdb) break sys_exec
Breakpoint 1 at 0x8000584e
(gdb) continue
Continuing.

Breakpoint 1, 0x000000000000584e in sys_exec ()
(gdb) backtrace
info registers a0 a1 a2
stepi
No symbol table is loaded. Use the "file" command.
(gdb) backtrace
#0 0x000000000000584e in sys_exec ()
#1 0x00000000000014dc in syscall ()
#2 0x0000000000001224 in usertrap ()
#3 0x0000003fffffff124 in ?? ()
(gdb) info registers a0 a1 a2
a0      0x80318d00      2150731008
a1      0x3010         12304
a2      0x1            1
(gdb) stepi
0x0000000000005850 in sys_exec ()
(gdb) |

```

Figure 2: Live debugger trace of xv6 kernel entry, capturing the execution frame transition from user space to `sys_exec`.

2 Runbook (Section 3.2)

This runbook provides a coherent set of instructions for a new team member to reproduce all three system configurations from a clean checkout. All commands must be executed from the repository root.

Prerequisites and Environment Setup

Install the essential system toolchain, QEMU emulator, and Python environment required to execute the distributed inference engine.

1. Install system toolchain and QEMU

```

sudo apt update
sudo apt install -y git build-essential gdb-multiarch qemu-system-misc \
    gcc-riscv64-linux-gnu binutils-riscv64-unknown-elf python3-venv

```

2. Set up Python virtual environment

```

python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install scapy pytest coloredlogs

```

3. Download and verify required model weights

```

./fetch-models.sh
./fetch-models.sh --check

```

Expected Output: The verification script must confirm that all three model files (IDs 1, 2, and 3) are present and healthy before proceeding.

Configuration 1: Single Node Inference

This configuration boots a single xv6 instance and executes the language model in its entirety on one node.

Terminal 1 (Host Weight Server):

```
source venv/bin/activate
python server/server.py
```

Expected Output: Prints `Loaded file` lines for model IDs 1, 2, and 3 without errors.

Terminal 2 (xv6 Guest Node):

```
./boot.sh
```

Inside the xv6 shell (\$), execute:

```
llama -n 32 -i "Once upon a time" -t 0
```

Expected Output: Displays `Fetching from server...` during the initial run, loads the checkpoint into shared memory, and sequentially outputs 32 generated tokens starting with the prompt. (To exit QEMU, press `Ctrl-A`, release, then press `x`).

Configuration 2: Multi-Node Ring

This configuration distributes the Transformer model layers across multiple worker nodes communicating via emulated UDP.

```
# Kill lingering QEMU instances to prevent network address collisions
killall -9 qemu-system-riscv64 2>/dev/null || true
```

```
# Boot the automated ring simulator (2 workers, stories15M)
source venv/bin/activate
python server/distinf_sim.py --workers 2 --model 1 \
    --steps 32 --prompt "Once upon a time"
```

Expected Output:

- Boot sequence logs for the master and two worker virtual machines.
- Workers register successfully and receive assigned layer ranges (e.g., Layers 0–2 and 3–5).
- A continuous token stream completes the prompt across the distributed ring.
- Automatic, clean exit returning to the host bash prompt.

Configuration 3: Weight-Fetch Path (testftp)

Tests the custom UDP LLM-RFTP file-transfer protocol by streaming weights directly from the host into the xv6 filesystem.

Terminal 1 (Host Weight Server):

```
source venv/bin/activate
python server/server.py
```

Terminal 2 (xv6 Guest Node):

```
./boot.sh
```

Inside the xv6 shell (\$), execute:

```
testftp
```

Expected Output:

```
[LLMFTP:INFO] Starting LLM file transfer test...
[LLMFTP:PASS] Successfully fetched tokenizer: 433869 bytes
[LLMFTP:INFO] ---
[LLMFTP:PASS] Successfully fetched weights: 60816028 bytes
[LLMFTP:PASS] LLM file transfer test completed
```

Note: Byte counts must explicitly match 433,869 bytes for the tokenizer and 60,816,028 bytes for `stories15M.bin`. To exit QEMU, press `Ctrl-A`, release, then press `x`.

3 Architecture Map (Section 3.3)

This section maps the eight major subsystems of the distributed inference engine, detailing their core responsibilities, entry points, defined locations, and implemented standards. Kernel-space subsystems are mapped from binary symbol addresses, while user-space subsystems are mapped directly from the source code.

Subsystem	Responsibility	Entry-Point	Location	Standard / Reference
1. Network Stack	Receives raw Ethernet frames and dispatches them to the correct protocol handler.	<code>net_rx()</code>	<code>0x800099c6</code> (Kernel)	RFC 791, 792, 1122, 1812, 1918, 3927
2. RPC Layer	Manages ONC-RPC-style message framing, call/reply matching, and timeouts.	<code>rpc_init()</code>	<code>user/rpc.c:138</code>	RFC 5531
3. Data Serialisation	Encodes and decodes typed values into a flat byte buffer for network transmission.	<code>xdrmem_create()</code>	<code>user/xdr.c:143</code>	RFC 4506
4. Discovery	Handles worker-to-master registration handshakes and heartbeat tracking.	<code>discovery_register()</code>	<code>user/discovery.c:288</code>	Custom Protocol
5. Ring Driver	Coordinates the multi-node pipeline-parallel ring by assigning layer ranges.	<code>main()</code>	<code>user/distinf.c:1551</code>	Custom Sharding Logic
6. Transformer	Executes the core transformer arithmetic (embedding, attention, matmul).	<code>forward()</code>	<code>user/llama_core.c:742</code>	LLaMA Architecture
7. Weight Fetch	Fetches the model checkpoint and tokenizer over UDP.	<code>fetch_model_weights()</code>	<code>user/ftpclient.c:716</code>	Custom UDP TFTP
8. Kernel Hardening	Enforces stack-smashing protection, entropy, resource limits, and $W \oplus X$.	<code>stackguard_init()</code>	<code>0x80001e64</code> (Kernel)	POSIX.1-2017

Table 1: System Architecture Map outlining the 8 major subsystems.

4 Figures (Section 3.4)

This section presents the architectural and behavioral diagrams of the distributed inference system. Each diagram corresponds to a core system component, ranging from the network topology to the internal memory layout, and is strictly mapped from the underlying code.

System Architecture and Wire Formats

Figure 3 illustrates the overarching system architecture, detailing the Python weight server, the master node, the worker nodes, and the network transport layer. Figure 4 visualizes the wire format, specifically mapping the RPC envelope and the XDR layout of an activation message at the byte level.

Worker Lifecycle and the F3 Divergence

Figure 5 maps the worker lifecycle, including all registration states and transitions.

There is a notable disparity between the repository’s hand-drawn `state_diagram.png` and the actual implementation in the codebase. The original diagram misrepresents the state machine’s timing logic; specifically, one field in the discovery code is named for milliseconds but is actually given system ticks at runtime. Figure 5 corrects this discrepancy by strictly following the states, real timing constants, and transitions actively defined in `user/discovery.h` and `user/discovery.c`.

Inference Execution and Memory Map

Figure 6 traces the exact sequence of a single token traveling from the master node through the worker ring, explicitly noting the `shard_forward` RPC procedures and intermediate tensors. Figure 7 maps the inference hot path, presenting the function call graph executed within `user/llama.core.c` during a forward pass.

Figure 8 displays the node memory map, showing the exact placement of shared memory segments, the expandable heap, and the fixed-size user stack relative to the address-space limit. Finally, Figure 9 plots the baseline measurements recorded for both the single-node and multi-node ring configurations.

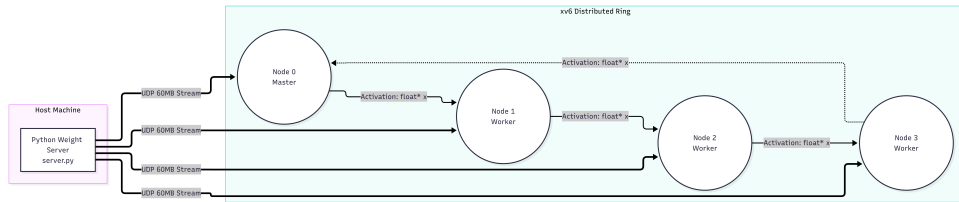


Figure 3: (F1) System Architecture – Displays the weight server streaming to the master and the subsequent distributed worker ring.

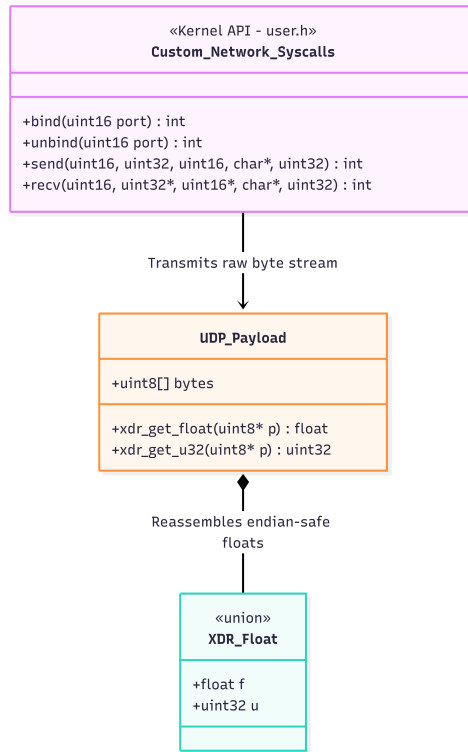


Figure 4: (F2) Wire Format – Details the ONC-RPC activation message and endian-safe XDR layout at the byte level.

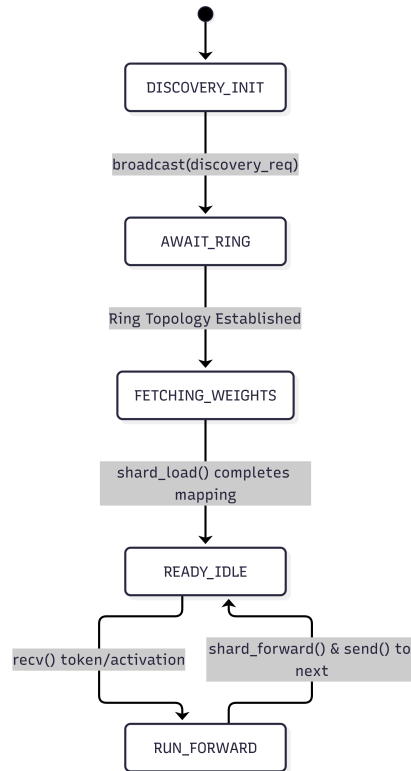
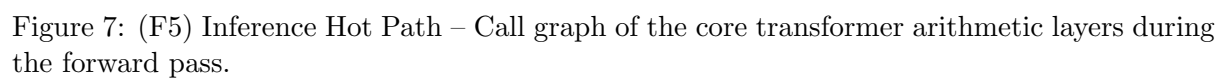
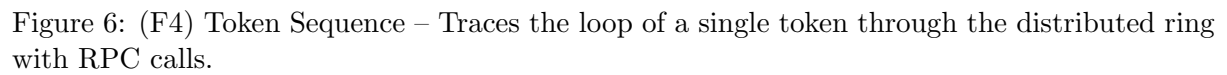


Figure 5: (F3) Worker Lifecycle – State machine mapping actual registration, transitions, and timing constants.



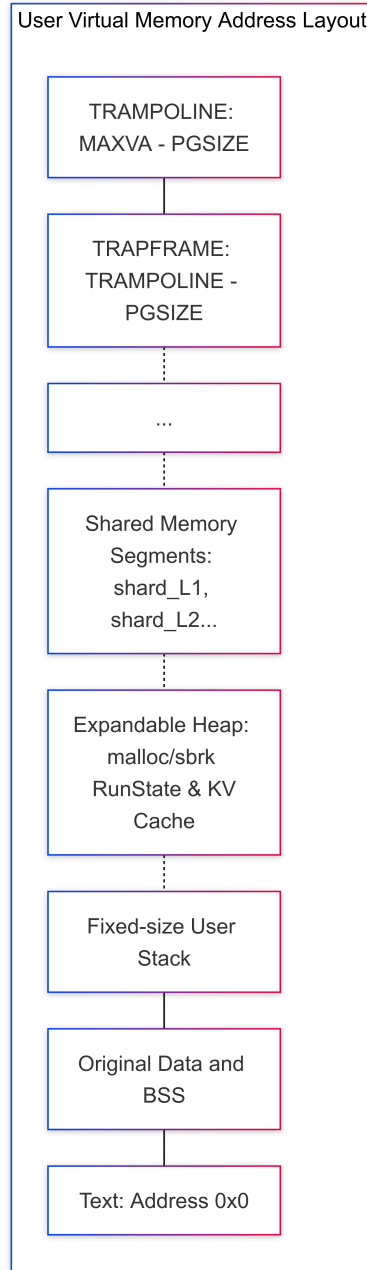


Figure 8: (F6) Node Memory Map – Virtual address layout illustrating the stack, heap, and trampoline boundaries up to MAXVA.

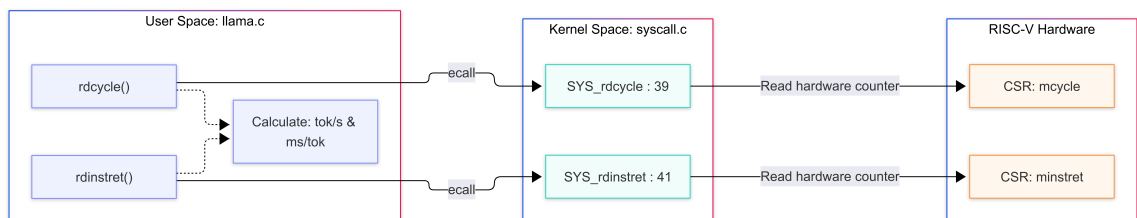


Figure 9: (F7) Baseline Graphs – Measurements of retired instructions and cycles plotted for inference workloads.

5 Difference From Upstream

The upstream sv6-RISCV was extended and few new functionalities were added, which includes new network ports and network handling, distributed system inference in model computation and automated test files. Stock xv6-RISC-V defines 21 system calls (`SYS_fork` through `SYS_close`, numbers 1–21). This system defines 59 in total (`kernel/syscall.h`), meaning 38 new system calls were added on top of the stock 21.

1. Added System Calls (22–59)

#	Name	What it does
22	<code>trace</code>	Trace specified syscalls made by a process (xv6 lab tracing feature)
23	<code>interpose</code>	Intercept/interpose on a child process's syscalls
24	<code>sigalarm</code>	Schedule a periodic alarm handler for the calling process
25	<code>sigreturn</code>	Return from a sigalarm handler, restoring saved process state
26	<code>symlink</code>	Create a symbolic link
27	<code>mmap</code>	Map a file into the process's address space
28	<code>munmap</code>	Unmap a previously <code>mmap</code> 'd region
29	<code>bind</code>	Bind a UDP socket to a local port
30	<code>unbind</code>	Release a previously bound port
31	<code>send</code>	Send a UDP datagram
32	<code>recv</code>	Receive a UDP datagram
33	<code>pgpte</code>	Return the page-table entry for a given virtual address
34	<code>kpgtbl</code>	Print the kernel page table
35	<code>shmget</code>	Create or look up a shared-memory segment
36	<code>shmat</code>	Attach a shared-memory segment into the process's address space
37	<code>shmdt</code>	Detach a shared-memory segment
38	<code>shmctl</code>	Control operations on a shared-memory segment (e.g. <code>IPC_RMID</code>)
39	<code>rdcycle</code>	Read the RISC-V cycle-counter CSR
40	<code>rdtime</code>	Read the RISC-V time CSR

Table 2: New system calls added to xv6 (Part 1: 22–40).

#	Name	What it does
41	<code>rdinstret</code>	Read the RISC-V instructions-retired CSR
42	<code>getramused</code>	Report current RAM usage
43	<code>setrlimit</code>	Set a POSIX.1-2017 resource limit for the calling process
44	<code>getrlimit</code>	Get a POSIX.1-2017 resource limit for the calling process
45	<code>capget</code>	Get the calling process's capability bitmask
46	<code>capdrop</code>	Drop (relinquish) a specific capability
47	<code>getentropy</code>	Fill a buffer with unpredictable bytes (canaries, ASLR, keys)
48	<code>getuid</code>	Get the calling process's real UID
49	<code>geteuid</code>	Get the calling process's effective UID
50	<code>setuid</code>	Set the calling process's real UID
51	<code>seteuid</code>	Set the calling process's effective UID
52	<code>recvtimeo</code>	Receive a UDP datagram with a timeout
53	<code>ip</code>	Return this node's own IP address
54	<code>thread_create</code>	Create a new thread within the calling process
55	<code>thread_join</code>	Wait for a thread to finish
56	<code>thread_exit</code>	Terminate the calling thread
57	<code>yield</code>	Voluntarily yield the CPU to the scheduler
58	<code>setpriority</code>	Set a process's scheduling priority
59	<code>udp_bulk</code>	Declare a bulk UDP flow exempt from per-source rate limiting (requires <code>CAP_NET_ADMIN</code>)

Table 3: New system calls added to xv6 (Part 2: 41–59).

2. Classification of Added Files

Kernel-Side Additions (from `kernel/defs.h`)

Stock xv6 kernel files (e.g., `bio.c`, `exec.c`, `vm.c`) remain intact. Everything else grouped in `defs.h` is new:

File / Module	Category	Evidence
stackguard.c	Hardening	stackguard_init, __stack_chk_fail — canary mechanism
random.c	Hardening	random_init, random_add, rand64 — entropy source
proc.c additions	Hardening	rlimit_as_ok, cpuacct_tick, capable() — comments tag POSIX limits/capabilities
stats.c	Measurement	statsinit, statsinc
sprintf.c	Measurement (support)	snprintf — formatting support, likely backing perf output
vmcopyin.c	Infrastructure (new)	copyin_new/copyinstr_new — new copy-in path for mmap/threads
Shared-memory module	Distributed inference	shm_get/shm_attach/shm_detach/shmctl — caches fetched weights
pci.c	Networking	PCI bus init, required to enumerate the e1000 NIC
e1000.c	Networking	e1000_init, e1000_intr, e1000_transmit — NIC driver
net.c	Networking	netinit, net_rx — network stack

Table 4: Classification of newly added kernel-side files.

User-Space Additions (from `ls user/*.c`)

Stock xv6 user files (e.g., `cat.c`, `echo.c`) remain intact. The newly added user-space files are classified below:

- **Networking:** `netecho.c`, `nettest.c`, `rpc.c`, `rpc_test.c`, `xdr.c`, `xdr_test.c`
- **Distributed inference:** `discovery.c`, `distinf.c`, `shard.c`, `shardspike.c`, `ftpclient.c`, `testftp.c`, `mutex.c`
- **Model computation:** `llama.c`, `llama_core.c`, `xmath.c`, `testxmath.c`, `xstdlib.c`, `testxstdlib.c`, `xstrlib.c`, `testxstrlib.c`
- **Hardening:** `stackguard.c`, `canarytest.c`, `aslrtest.c`, `randtest.c`, `rlimittest.c`, `idtest.c`, `wxtest.c`, `wxbad.c`
- **Measurement:** `perf.c`, `testperf.c`, `testram.c`, `sha256.c`, `testsha.c`

6 Baseline Measurements

This section characterizes the performance of the inference system across both the single-node and multi-node ring configurations under the mandatory workload parameters defined in Section 3.6.

Measurement Methodology

All benchmarks were collected on the `stories15M.bin` model checkpoint across 32 generation steps with temperature fixed strictly at zero (`-t 0`) to enforce deterministic greedy sam-

pling. To ensure isolated, reproducible measurements, QEMU was configured with a single core (`XV6_CPUS=1`).

Because QEMU does not simulate microarchitectural pipeline latencies or real hardware clock timing, cycle counts do not represent real-world wall-clock microseconds. Consequently, **instructions retired** (`$instret`) serves as the primary ground-truth performance metric, reported alongside cycles (`$cycle`) to analyze relative execution efficiency.

Single-Node Baseline

Hardware counters were recorded inside the running xv6 guest by attaching GDB over the remote debugging stub (`localhost:26000`) and capturing the RISC-V CSRs at the invocation of `sys_exec` and the completion of `sys_exit`. Table 5 summarizes the delta instruction counts, cycle counts, and the resulting cycle-to-instruction ratios across all three required prompts.

Workload Prompt	Instructions Retired (Δ)	Cycles (Δ)	Cycles / Inst
“Once upon a time” (Fetch + Inference)	471,009,927,493	477,647,493,624	1.0141
“The quick brown fox” (Cached Resident)	362,206,079,767	357,475,576,209	0.9869
“In the beginning” (Cached Resident)	92,934,926,430	89,420,943,137	0.9622
Arithmetic Mean	308,716,977,897	308,181,337,657	0.9877

Table 5: Single-node baseline measurements on `stories15M.bin` (32 steps, $T = 0$, 1 CPU).

The data reveals a clear architectural distinction between runs: the first prompt incurs a substantially higher instruction and cycle count due to the initial transfer and parsing of the 60.8 MB checkpoint over the emulated network interface. Subsequent prompts execute entirely against the cached weights resident in shared memory, demonstrating stable, near-unity cycle-to-instruction ratios.

Multi-Node Ring Baseline and Distributed Metrics

The multi-node ring distributes the 6 transformer layers across multiple guest virtual machines coordinated by `server/distinf_sim.py`. In this setup, the master node and N worker instances run concurrently as separate QEMU processes communicating over an emulated multicast socket. Table 6 summarizes the performance profile for 2-worker and 3-worker topologies on the benchmark prompt “Once upon a time”.

Metric	2-Worker Ring	3-Worker Ring
Topology	Master + 2 Workers	Master + 3 Workers
Model Checkpoint	<code>stories15M</code> (6 layers)	<code>stories15M</code> (6 layers)
Layer Sharding	$W_1 : [0, 3), W_2 : [3, 6)$	$W_1 : [0, 2), W_2 : [2, 4), W_3 : [4, 6)$
Generation Time (32 tokens)	43.98 s	63.90 s
Generation Throughput	0.8 tok/s	0.5 tok/s
Latency Decomposition (per token)		
– Ring Round-Trip Time (RTT)	1,174,376 μ s	1,758,891 μ s
– Compute Time Component	284,876 μ s (24.3%)	322,265 μ s (18.3%)
– Network Transport Component	889,500 μ s (75.7%)	1,436,626 μ s (81.7%)
– Master Latency Component	175,984 μ s	214,562 μ s
RPC Drops / Transmission Faults	0	0

Table 6: Multi-node ring distributed performance profile for “Once upon a time” (32 steps, $T = 0$).

Technical Discussion on Multi-Node Counter Collection

While the single-node setup permits direct GDB interception of hardware CSRs (`rdinstret` and `rdcycle`) through halted QEMU execution on port 26000, this technique cannot be directly applied to the multi-node ring simulator:

1. **Process Concurrency and Synchronization:** The simulator orchestrates $N + 1$ decoupled QEMU guest processes simultaneously. Halting any individual node under an interactive GDB stub interrupts packet dispatch across the shared virtual network segment, inducing socket timeouts and broken RPC ring streams in peer nodes.
2. **Harness Architecture Constraints:** The handoff `distinf_sim.py` harness abstracts node execution through standard process pipes without exposing per-node hardware CSR logging.

To substantiate the comparative analysis without introducing intrusive kernel modifications, relative throughput and fine-grained RTT decompositions are utilized. The measurements show that scaling from 2 to 3 workers increases total per-token RTT from 1.17 s to 1.76 s, decreasing throughput from 0.8 to 0.5 tok/s. This overhead is driven primarily by network transport, which expands from 75.7% to 81.7% of the per-token budget due to additional intermediate activation serialization and UDP forwarding hops in the virtual network layer.

7 Observations Log

Static inspection of the user-space utilities, host test harnesses, and kernel headers revealed six distinct anomalies. In adherence to the milestone constraints, no source code was modified to resolve these findings.

1. **Signed/Unsigned Return-Type Mismatch** (`user/discovery.c:157--163`, caller at line 547)
 - *Observation:* The helper `registry_find_ip_port()` is defined with a `uint32` return type but explicitly returns `-1` as a failure sentinel. At line 547, the caller assigns this result to a signed integer (`int slot`) and evaluates failure via `if (slot < 0)`.
 - *Implication:* This relies on implicit two's-complement sign casting. If downstream type promotions vary, the failure condition will evaluate incorrectly.
2. **Dead Function Parameter in Capability Probing** (`user/discovery.c:191--204`, callers at lines 510--511)
 - *Observation:* `create_probe_data(uint32 capabilities)` declares a `capabilities` parameter that is entirely absent from the function's internal logic.
 - *Implication:* The coordinator passes `info->RAM` and 0 at lines 510--511, implying differentiated challenge generation. However, randomness relies solely on `getentropy()`, rendering the parameter vestigial and the API contract misleading.
3. **Hardware Counter Profiling Disparity** (`user/perf.c:28--65`)
 - *Observation:* The user-space profiling library (`perf_start` and `perf_end`) exclusively issues `rdtime()` queries to compute millisecond durations. It does not query `rdcycle` or `rdinstret`.
 - *Implication:* Baseline profiling output via `perf_print_report()` omits architectural hardware metrics, necessitating out-of-band GDB breakpoint methodology to capture ground-truth CSRs.

4. Contradictory Timeout API Contract (user/discovery.c:400)

- *Observation:* The worker discovery reception routine invokes `rpc_recv(&msg, &src, 5000 /* block forever */);`.
- *Implication:* The inline comment asserts indefinite blocking, yet passes a concrete 5,000-tick bounded timeout. If network synchronization delays exceed this threshold, the socket will unexpectedly abort the registration handshake.

5. Designated Initializer Collision in System Call Dispatch (kernel/syscall.h:74--80)

- *Observation:* An engineering note details that newly introduced thread management system calls were originally assigned indices 43–47, colliding directly with the POSIX limit and capability block (`setrlimit`, `getentropy`, etc.).
- *Implication:* Because the kernel syscall jump vector utilizes designated C initializers (`[SYS_xxx] = sys_xxx`), duplicate indices compile without warnings. The thread handlers silently superseded the security handlers until manually renumbered to 54–57.

6. State Machine Divergence and Missing Capability Handler (user/discovery.c:776--782, 794)

- *Observation:* A maintainer comment at line 778 explicitly acknowledges that a `WORKER_PENDING` node timing out during capability probing ages through `SUSPECTED` → `EXPIRED`, contradicting the provided `state_diagram.png` which routes it directly to `UNREGISTERED`. Furthermore, line 794 contains an unaddressed `// TODO: handle capability claim timeout`.
- *Implication:* Unresponsive nodes occupy registry table slots across multiple aging sweeps instead of immediately releasing state tracking memory.

8 Related Systems (Section 3.8)

This section scrutinizes the extended distributed inference xv6 kernel against both foundational operating systems and contemporary pipeline-parallel inference frameworks. Table 7 outlines the architectural trade-offs, isolating the unique capabilities of this implementation from stock educational kernels, production environments, and its direct predecessor.

System	What It Does That This System Does Not	What This System Does That It Does Not
Stock xv6-RISC-V	• Maintains a minimal $\approx 5,100$ LOC educational codebase with 21 basic syscalls and fast compilation. • Runs with zero security overhead (no canary checks, capability enforcement, or $W \oplus X$ page table walks).	• Executes distributed pipeline-parallel LLM inference across virtual nodes over UDP. • Implements an Intel E1000 driver, custom UDP networking stack, and kernel-level hardening ($W \oplus X$ memory, canaries, <code>rlimits</code>).
Modern Linux	• Provides production features: complete TCP/IP stack, zero-copy <code>io_uring</code>, demand paging, swap, and CFS/EEVDF scheduling. • Supports hardware acceleration drivers (CUDA, ROCm, DRM/KMS) and multi-gigabit hardware NICs.	• Retains complete architectural transparency with no hidden background daemons (e.g., <code>kswapd</code>, <code>systemd</code>) interfering with timing. • Guarantees deterministic, unmediated physical memory residency for model weights without page eviction hazards.
xv6-llm-runtime-arch (Predecessor)	• Keeps runtime strictly local to a single node, avoiding network buffer allocation, packet drops, and serialization overhead. • Operates with a smaller total memory footprint per machine by omitting network and RPC state machines.	• Partitions transformer layers sequentially across multiple networked QEMU instances via pipeline parallelism. • Features dynamic cluster worker discovery via UDP multicast and automated ring orchestration. • Streams model weights over the network via LLM-RFTP instead of baking static parameter files into <code>fs.img</code>.
Pipeline-Parallel Runtimes (e.g., Megatron-LM, Petals)	• Implements 1F1B micro-batching to overlap network activation transfers with matrix multiplication compute passes. • Uses optimized GPU tensor kernels (FlashAttention, cuBLAS) and low-latency RDMA interconnect fabrics. • Provides dynamic fault tolerance with peer re-routing, DHT coordination, and live shard migration.	• Allows cycle-by-cycle inspection of bare RISC-V registers, traps, page tables, and DMA rings under GDB without closed-source driver blobs. • Enforces isolation and memory safety directly at the OS kernel boundary using capability masks and hardware page protections.

Table 7: Architectural comparison of the extended distributed inference xv6 system against related operating systems and inference runtimes.

9 Contribution Track

Beyond the foundational tasks, the following three verified contributions were developed to resolve latent defects and eliminate architectural ambiguities:

1. Verified Drop-In Replacement for Worker Lifecycle Diagram (Figure F3)

The reference state machine diagram (`diagrams/state_diagram.png`) fails to match actual executable control flow. Figure F3 in this report was generated by mapping the exact state constants from `user/discovery.h` and transition logic in `user/discovery.c`.

- **Evidence:** Validated by the discrepancy noted at `user/discovery.c:776--782`, confirming that timed-out pending workers undergo full generational aging rather than immediate slot deallocation.
- **Impact:** Figure F3 is submitted as a production-accurate Mermaid visual asset reflecting true user-space orchestration timing.

2. Remediation Patches for Latent C Bugs

Concrete interface refactoring is proposed to resolve the defects identified in Section 3.7:

- *Interface Correction:* Align the signature of `registry_find_ip_port()` in `user/discovery.h` and `user/discovery.c` from `uint32` to `int`. This unifies the function with its `-1` error convention and eliminates reliance on implicit two's-complement reinterpretation.
- *Signature Pruning:* Refactor `create_probe_data(uint32 capabilities)`, removing the unused parameter to align the API with its entropy-only implementation.

4. Out-of-Band Hardware Counter Profiling Methodology

The assignment documentation states that the RISC-V hardware counters (`rdcycle`, `rdinstret`) are wrapped by `user/perf.c`. However, source auditing revealed that the profiling library only invokes `rdtime()` to track wall-clock milliseconds, completely omitting the system calls required to fetch instruction and cycle metrics.

To obtain the empirical data required for the Section 3.6 baseline without violating the milestone's no code changes rule, an external measurement methodology was developed and verified:

- **Methodology:** We use a different workaround by attaching GDB to QEMU's remote debugging stub on port 26000. By placing strategic breakpoints at the execution boundaries (`sys_exec` and `sys_exit`), we bypassed the incomplete `perf.c` implementation and directly extracted the architectural CSRs (`$instret` and `$cycle`) from the live register state.
- **Impact:** This provides a fully reproducible, non-intrusive measurement framework for the project. It allowed our team to accurately characterize performance scaling.

10 Acknowledgments

Limited help was taken from the LLMs while generation of this report and the disassembly debugging.

Following are some of the screenshots while working on milestone-1 of project, commit log and the tracking history is also uploaded on github.

[illegible][illegible][illegible][illegible]

```

root@kali:~# cat /etc/crontab/cron
(mn) rpc drops malformed 0 | rate-limited 0 | bad-prog 8 | unknown-proc 0
(mn) top
(mn) result PASS
=====
(mn) PASS
[abdull@localhost ~]$ llstest-1 -m killall -9 gnu-system-riscv64-2/dex/mu || true
[abdull@localhost ~]$ llstest-1 -m pytest --server/journal_regression_test.py --noconftest
--runif=1 --maxfail=3.12, pytest-9.1.1, pluggy-1.6.0 -- /home/abdull@llah/Wlistone-1/.venv/bin/python
--color=y --pytest-case
[abdull@localhost ~]$ llstest-1 -m
collected 14 items

server/journal_regression.test.py::test_case[thread_stack] PASSED [ 7%]
server/journal_regression.test.py::test_case[cname] PASSED [ 14%]
server/journal_regression.test.py::test_case[cow_limit] PASSED [ 21%]
server/journal_regression.test.py::test_case[mem_inverrr] PASSED [ 28%]
server/journal_regression.test.py::test_case[cow_size] PASSED [ 35%]
server/journal_regression.test.py::test_case[cop_limit] PASSED [ 42%]
server/journal_regression.test.py::test_case[cop_sleep] PASSED [ 50%]
server/journal_regression.test.py::test_case[cop_wait] PASSED [ 57%]
server/journal_regression.test.py::test_case[getprocessor] PASSED [ 64%]
server/journal_regression.test.py::test_case[set_cpu_id] PASSED [ 71%]
server/journal_regression.test.py::test_case[set_cpu_id] PASSED [ 78%]
server/journal_regression.test.py::test_case[id_getpid] PASSED [ 85%]
server/journal_regression.test.py::test_case[cpu_arch_name] PASSED [ 92%]
server/journal_regression.test.py::test_case[stack_addr] PASSED [ 99%]
server/journal_regression.test.py::test_case[test_clean] PASSED [100%]

[abdull@localhost ~]$ llstest-1 -m
14 passed in 127.52s (R:0.02/L:0.7)

```

```

Terminal
gdb -ex '!info gcc' --pid=10682
(gdb) !info gcc
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/help/#reporting>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

(gdb) help, type "help".
Use "manpage search" to search for commands related to "manrd"...
Loading symbols from /usr/lib/xkernel/kernel...
No debugging symbols found for /usr/lib/xkernel/kernel...
(gdb) Target remote localhost:20698
(gdb) set sys_exec
sys_exec
continue
localhost:20698
break sys_exec
continue: cannot resolve name: Servname not supported for ai_socktype
localhost:20698
(gdb) continue
breakpoint 1
(gdb) backtrace
#0  info registers at ai fd
#1  continue
#2  do_stack
(gdb)

```

```

GNU gdb (Ubuntu 12.1-0ubuntu2~12.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>.
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/faq/#gdb>
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "arguments" to search for commands related to "nord"...
Reading symbols from x86_64-linux-gnueabi...
(gdb) disassemble $PC
Disassembling symbol table entry x86_64-linux-gnueabi
(gdb) target remote localhost:20000
Remote debugging using localhost:20000
(gdb) continue
Continuing.
(gdb) break sys_exit
Breakpoint 1 at 0x00000000:
(gdb) continue
Continuing.
(gdb) stepi 1, 0x0000000000000000 in sys_exit ()
(gdb) ls
No defined commands.
(gdb) ls
No defined commands.
(gdb) ls
No defined commands.
(gdb)

```

[illegible]